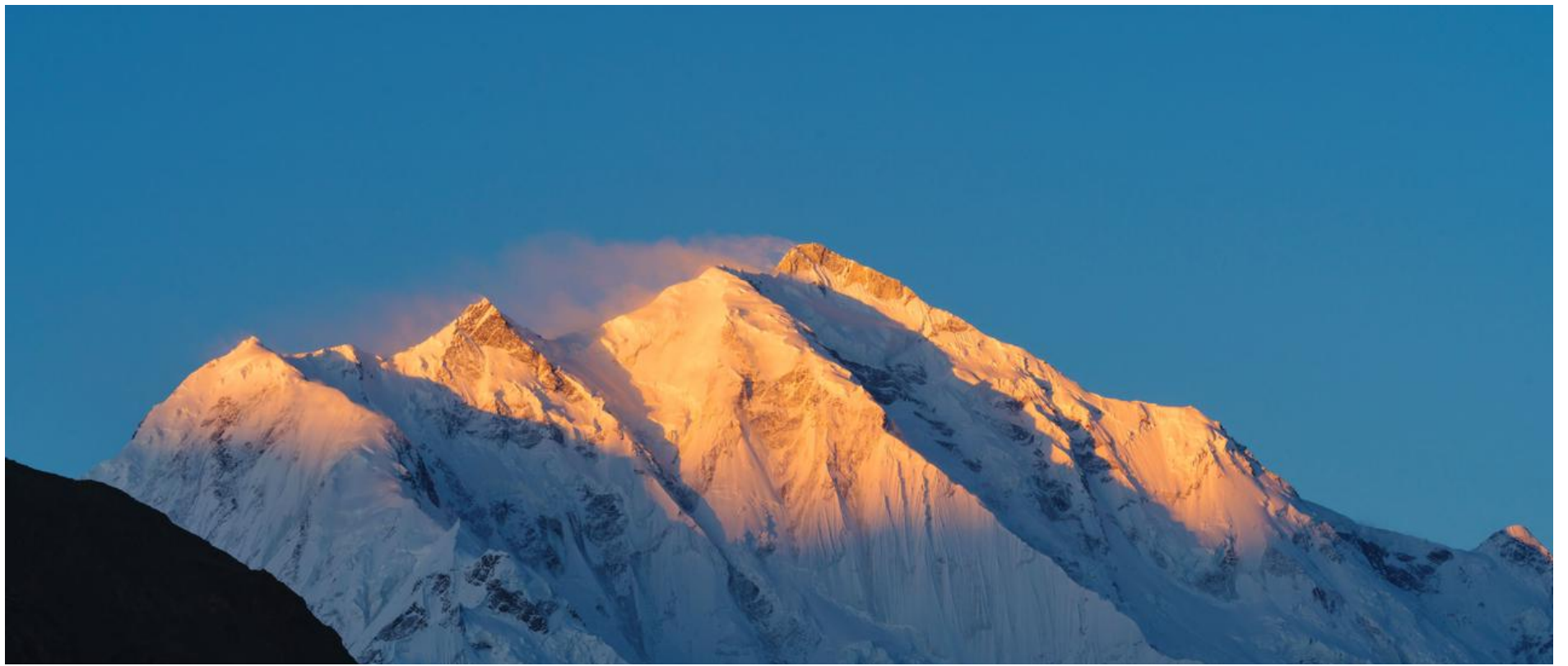




Web性能手术刀：前端性能优化系统性讲解（上）

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 -- Typescript: JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)

2. 简历

作为面试的敲门砖，大/中/小厂的简历分别有什么异同点？如果在简历面上胜人一筹？

2.1 简历内容详细分析

简历模板的选择很讲究，有些简历基本不看内容就会被刷掉，这些简历一般会对面试官进行视觉攻击，让简历给面试官的第一印象就是反感。

有两种**令人反感的简历模板**：

1. 一种是经典简历模板，真是堪称经典，配色比较魔幻，加上表格类的简历属于low到底端的简历类型，基本上扫一眼就扔了，这种简历只需要3秒钟就能被面试官扔到垃圾堆。
2. 另一种是设计感十足的简历模板，这种简历设计感十足，这五颜六色的配色能亮瞎面试官的双眼，这种花里胡哨的简历同样也是3秒钟沉到垃圾堆底部的简历。

以上两类简历模板堪称面试官杀手，只要用了上述两类模板，绝对连让面试官看第二眼的兴趣都没有。面试官筛简历要的是**高效、清晰、内容突出**，不管是HR还是技术面试官都想在最快速的情况下看到有效信息，你眼中所谓的『视觉效果』在别人眼里就是『视觉噪音』或者『视觉垃圾』，严重影响看简历的心情和寻找有效信息的速度。

2.1.1 准备个人信息

个人信息部分主要包括**姓名、电话、电子邮箱、求职意向**，当然这四个是必填的，其它的都是选填，填好了是加分项，否则很可能减分。

接下来才是重点：

- 1.**博客**：如果准备一个基本没有更新的博客或者没有任何贡献的github，那么给面试官一种为了放上去而放上去的感觉，这基本上就是在跟面试官说『**这个候选人平时根本没有总结提炼的习惯**』，所以如果有长期维护的github或者博客一定要放上去，质量好的话会非常有用，如果没有千万别放。
- 2.**学历**：如果你的学历是专科、高中毕业之类的，还写在简历头部强调一遍，这就增加了简历被刷的几率，如果是研究生或者985211学历可以写，突出一下学历优势，本科学历在技术面试领域基本上敲门砖级别的，没必要写。
- 3.**年龄**：如果是大龄程序员，尤其是还在求一份低端岗位的时候千万别写，一个大龄程序员在求职一个中低端岗位，说明这些年基本原地踏步，还不能加班，到这里基本上此简历就凉了一半了。
- 4.**照片**：形象优秀的可以贴，尤其是形象优秀的女程序媛，其它的最好不要贴，如果要贴的话，最好是贴那种PS过的非常职业的证件照。

2.1.2 准备专业技能

对于程序员的专业技能其实就是技术栈，对于自己的技术栈如何描述是个很难的问题，比如什么算是精通？什么算是了解？什么是熟悉？

关于对技术技能的描述有很多种，这里只讲三种的了解、熟悉、精通：

- 了解**：使用过某一项技术，能在别人指导下完成工作，但不能胜任复杂工作，也不能独立解决问题。
- 熟悉**：大量运用过的某一项技术，能独立完成工作，且能独立完成有一定复杂度的工作，在技术的应用层面不会有太大问题，甚至理解一点原理。
- 精通**：不仅可以运用某一门技术完成复杂项目，而且理解这项技术背后的原理，可以对此技术进行二次开发，甚至本身就是技术源码的贡献者。

以Vue框架为例，如果可以用Vue写简单的页面，单独完成某几个页面的开发，但是无法脱离公司脚手架工作，也无法独立从0完成一个有一定复杂度的项目，只能称之为了解。

如果有大量运用vue的经验，有从0独立完成一定复杂度项目的能力，可以完全脱离脚手架进行开发，且对vue的原理有一定的了解，可以称之为熟悉。

如果用vue完成过复杂度很高的项目，而且非常熟悉vue的原理，是vue源码的主要贡献者，亦或者根据vue源码进行过魔改（比如mpvue），可以称得上精通。

那么有两个坑是候选人经常犯的，**杂和精**，这种两个坑大量集中在应届生和刚毕业没两年的新手身上，其主要特点是急于表现自我、对技术深度与广度出现无知而导致的过度自信。

首先说杂，比如要应聘一个Vue前端，老老实实把自己的Vue技术栈写好就行了，强调一下自己擅长什么即可，最好专精某领域比如后台管理系统实践等。这时一些简历非要给自己加戏，自己会的不会的一股脑往上堆，什么工程化定制、各个系统从0到1落地、AI都要体现出来，越杂越好，这种简历给人的印象就是个什么都不懂的半吊子。

再说精，一个刚毕业的应届生，出来简历就各种精通，精通Vue、精通Vue源码、精通React源码等等，这种简历是不会没头没脑投过来的，这种在大学里就精通各种的天才早被他的各种学长介绍进了大厂或者外企做某某Star重点培养了，往往看到的这种也是半吊子。

2.1.3 准备工作经历

工作经历本身不用花太多笔墨去写，面试官主要想看的就是每段工作经历的持续时间、在不同公司担任的职责如何、是否有大厂的工作经验等。

那么什么简历在这里给面试官减分呢？

●**频繁跳槽**：比如三年换了三家公司，每个公司呆的时长不要超过一年；

●**常年初级岗**：比如工作五六年之后依然在完成一些简单的项目开发；

●**末流公司经历**：在技术招聘届，大厂的优先级最高比如BAT、TMD甚至微软、谷歌等外企，知名度独角兽其次，一般的互联网公司排在第三，就是工作中小型的互联网公司一般大家叫不上名字，排在最后的就是外包和传统企业的经历；

(1) 所以，如果你有频繁跳槽的经历怎么办？在本公司老老实实等到满一年再跳槽。

(2) 如果常年初级岗怎么办？想办法晋升或者参与一些业界知名项目，再或者写一个有一定复杂度的私人项目。

(3) 如果有末流公司经历怎么办？如果是很久以前的末流公司经验可以直接不写，也没人在乎你很早之前的工作经历，如果你现在就在末流公司，赶紧想办法跳槽，去不了大厂，去非知名的互联网公司也算是胜利大逃亡了。

2.1.4 准备项目经历

项目经历不管对于社招还是校招都是重中之重，很多时候成败就在于项目经历这块，一个普通本科可以通过优秀的项目经历逆袭985，一个小厂的员工也可以获得大厂的面试机会。但是必须要说一下项目经历的编写很讲究，这是为后面面试部分铺路的绝佳机会，也是直接让你的简历扑街的重点沦陷区域。

先来看简历中项目经历的坑。

2.1.4.1 切忌流水账写法

项目经历流水账写法是绝大多数简历的通病，通篇下来就讲了一件事『我干了啥』。

大部分简历却是这样的：

用Vue、Vuex、Vue-router、axios等技术开发电商网站的前端部分，主要负责首页、店铺详情、商品详情、商品列表、订单详情、订单中心等相关页面的开发工作，与设计师与后端配合，可要高度还原设计稿。

这个描述有什么问题？其实看似也没啥问题，但是这种流水账写法太多了，完全无法突出自己的优势展现自己的能力。**项目经历是考察重点，面试官想知道候选人在一次项目经历中扮演的角色、负责的模块、碰到的问题、解决的思路、达成的效果以及最后的总结与沉淀。**

而上面的描述只显示了我干了啥，所以这种项目描述几乎是没意义的，因为对于面试官而言他看不到有效信息，没有有效信息的项目描述基本就没价值了，如果这个时候你还没有大厂经历或者名校背书，基本上也就凉了。

2.1.4.2 切忌堆积项目

堆积项目这种现象往往出现在没有什么优秀项目经历的简历身上，候选人企图以数量优势掩盖质量的劣势，其实往往适得其反，项目经历的一栏最好放2-3个项目，非常优秀的项目可能放一个就足够了，举个极端例子如果有一天尤雨溪写简历，其实只需要在项目经历那些一行Vue.js作者就行了,当然,他并不需要投简历。

有一些项目切忌放上去：

●**demo级项目**：很多简历居然还在放一些仿xx官网的demo，这是十足的减分项，有一些则是东拼西凑抄了一些框架的源码搞了个玩具项目，没有任何价值。

●**烂大街的项目**：这种以vue技术栈的为最，由于视频网站的某门课程流行，导致大量的仿饿了么、仿qq音乐、仿美团、仿去哪儿，十份简历5份一模一样的项目，你是面试官会怎么想。

●**低质量的开源项目**：一个大原则就是低star的尽量别放（除非是高质量代码的冷门项目），长期弃坑的也不要放，不要为了凑数量把低质量的项目暴露出来，好好藏着。

如果只放两个项目，最好的搭配是一个公司内部挑大梁的项目和一个社区内的开源项目，后者之所以可以占据一席之地，是因为通过你的开源项目，面试官可以通过commit完整看到你的创造过程，比如工程化建设、commit规范、代码规范、协作方式、代码能力、沟通能力等等，这甚至比面试都有用，没有比开源项目更能展示你综合素质的东西了。

2.1.4.3 切忌放虚假项目

一个项目做没做过只要有经验的面试官一问便知，如果你真的靠假项目忽悠过了面试，那这个公司八成也有问题，人才把关不过硬，你可以想象你的队友都是什么水平，在这种公司大成长价值也不大。好，如果你说实在没项目可写了，我只能造假了，那么你应该想一下这多层追问。

比如你说你优化了一个前端项目的首屏性能，降低了白屏时间，那么面试官对这个性能优化问题会进行深挖，来考察候选人的实际水平：

- 你的性能优化指标是怎么确定的？平均下来时间减短了多少？
- 你的性能是如何测试的？有两种主流的性能测试方法你是怎么选的？
- 你是根据哪些指标进行针对性优化的？
- 除了你说的这些优化方法还有没有想过通过xx来解决？
- 你的这个优化方法在实际操作中碰到过什么问题吗？有没有进一步做过测试？
- 我们假设这么一种情况，比如xxxx，你会这么进行优化？

面试官多层追问的逻辑是这样的：**了解背景 -> 了解方案 -> 深挖方案 -> 模拟场景**

首先得了解你性能优化的指标如何，接着需要了解你是这么测试的指标、再怎么进行针对性优化的，再接着提出一些其它解决方案考察你对优化场景的知识储备和方案决策能力，最后再模拟一个其它的业务场景，来考察你的技能迁移能力，看看是否是对某块领域有一定的了解，而不是只针对某个项目。

如果要真的在面试现场对答如流，那么一定是在某一块领域有一定知识储备的人，不是随随便便搞个项目就能蒙混过关的。

2.1.4.4 合格的项目经历

合格的项目经历必须要有以下几点：

- 项目概述
- 个人职责
- 项目难点
- 工作成果

如果不怕字太多，还可以选择性加入解决方案、选型思路等，但是由于篇幅限制和为面试铺垫就不太建议写得太多。

项目概述的目的是让面试官理解项目，不是每个人面试官都做过你的那种项目，所以需要个简述方便面试官理解。

个人职责就是告诉面试官你在本项目中扮演的角色，是领导者？主导者？还是跟随者，你负责了哪些模块，承担了多大的工作量，以此来评估你在团队中的作用。

项目难点的目的在于让面试官看到你碰到的技术难题，方便后续面试对项目进行一系列讨论。

工作成果就很明显了，面试官需要看到你在做了上述工作到底达成了什么成绩，这个时候最好以数据说话，比如访问量、白屏时间等等。

这个时候也切忌长篇大论，把技术细节都写上去，甚至还写了心路历程的都是大忌，一方面篇幅太大会造成视觉混乱，另一方面面试官想看到的是简历，不是技术总结，面试官要面对上百份简历没那么时间来看你长篇大论，长篇大论大可以在面试中展开。

最好的方法就是一行文字简单得说清楚即可，反正项目面的时候会问到，到时候把准备的内容讲给面试官，掌握面试的主动权就是从项目经历这一栏中开始。

2.1.4.5 其他

- 教育背景

应届生可以写得更详细一点，比如绩点排名怎么样，有没有突出的科目，社招就不要写太多了，简单的入学时间、学校、专业即可，而且写最高学历即可，没必要从初中就开始写学历流水账，没有人看的。

• 注意事项

自我评价不建议写：技术面试几乎没人看你的自我评价，连面试技术问题都嫌『talk is cheap show me the code』，你的自我评价除了占篇幅没啥用处，充其量算是面试官的干扰信息。

• 简历封面千万别搞

这都是一些简历制作网站骗用户付费的伎俩，不仅是互联网行业，其它行业我也没见过要简历封面这种无用操作的。

• 证书不建议写

应届生可以酌情考虑弄个六级证书什么的，对于社招而言，列一堆证书甚至是减分项，国内的各种证你也懂的，是有多不自信才沦落到靠一堆证书来证明自己的价值。

• 千万别用技能图表

首先用90分、80分来评价自己的技术本身就没有什么说服力，也不可能这么精准，而且什么是90分、什么是80根本就没有一个公论，所以用一般的比较通用的熟悉、精通描述即可，千万别加戏，面试官或者HR没那么多闲工夫去理解你的图表，老老实实按最通用高效的方式描述自己的技术栈。

• 简历最好不超过2页

程序员又不是设计师有时候需要作品呈现，如果简历超过一页一定是出问题了，项目、技术栈描述太多太杂占据了大量篇幅。

2.2 真实简历分析&优化

飞书链接：[📄 13. 如何写出高质量的简历（For 付费学员）](#) 密码：86t&9493

3. 具体技术栈应该掌握到什么程度？



印客2025Web前端大厂训练营.pdf
19.34MB



4. 性能优化课程资料



about-optimize.zip
1.87MB



5. 为什么前端性能如此重要

前端性能直接影响用户体验，它关系到用户是否愿意继续使用你的应用。根据Google的研究，页面加载时间每增加0.1秒，转化率就会下降7%。慢速的网站不仅会影响用户体验，还会影响搜索引擎排名，从而影响网站的可见性和业务目标。

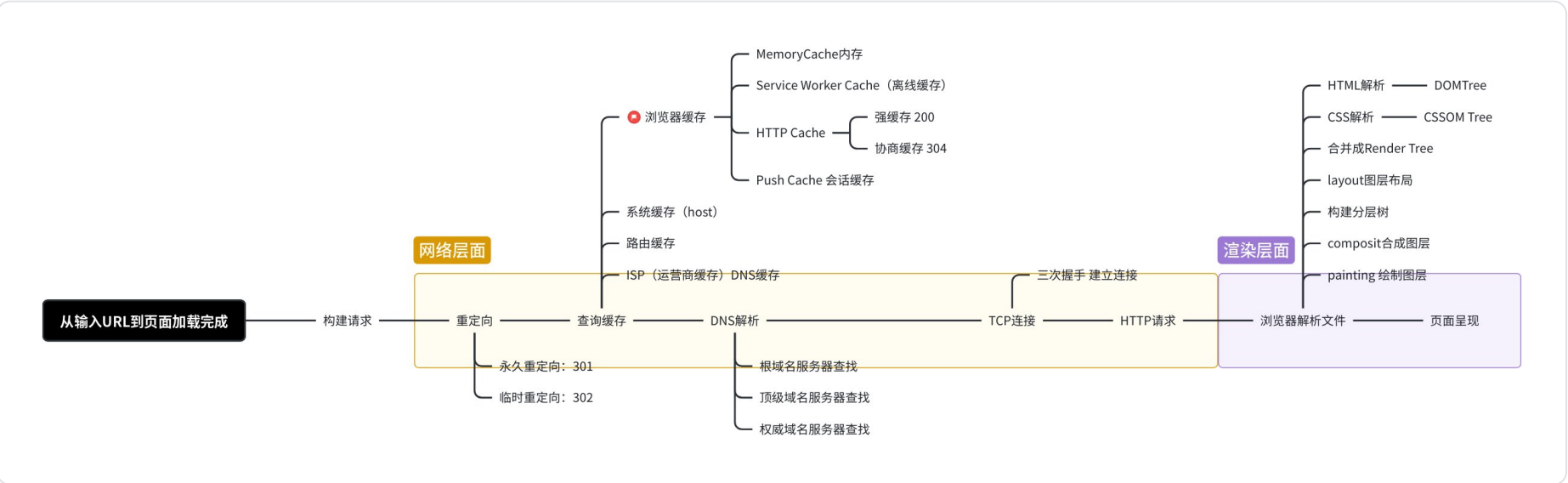
性能对用户的影响

- **留存率**：慢速网站会导致用户流失，53%的移动用户会放弃加载时间超过3秒的页面
- **转化率**：页面加载速度与转化率呈负相关，每延迟1秒，转化率可能下降7%
- **品牌形象**：性能卓越的产品会给用户留下专业、可靠的印象
- **用户满意度**：流畅的体验会提升用户满意度和使用黏性

6. 从输入 URL 到页面加载完成，发生了什么？

这是老生常谈的面试问题，我们的性能优化也将站在这个角度谈起。

首先我们需要通过 DNS（域名解析系统）将 URL 解析为对应的 IP 地址，然后与这个 IP 地址确定的那台服务器建立起 TCP 网络连接，随后我们向服务端抛出我们的 HTTP 请求，服务端处理完我们的请求之后，把目标数据放在 HTTP 响应里返回给客户端，拿到响应数据的浏览器就可以开始走一个渲染的流程。渲染完毕，页面便呈现给了用户，并时刻等待响应用户的操作（如下图所示）。



1. 构建请求：根据 url 构建请求
2. 重定向：永久重定向、临时重定向

特性	永久重定向（301）	临时重定向（302/307）
HTTP 状态码	301	302（旧）、307（新）
SEO 影响	旧 URL 权重转移至新 URL	保留旧 URL 权重
浏览器缓存	长期缓存	不缓存或短期缓存
用途	永久性变更（如域名迁移）	临时性跳转（如维护、测试）
请求方法保留	可能变为 GET（历史实现）	307 保留原始请求方法

3. 查询缓存：浏览器缓存、系统缓存、路由缓存、DNS缓存查找
4. DNS解析：根域名、顶级域名、权威域名递归查找对应的ip地址
5. TCP连接：传输层 TCP 三次握手建立连接
6. HTTP请求：应用层 client 端向 server 端发起 HTTP 请求报文，server端收到请求处理后，响应 client 端返回响应报文（html、js、css等资源）
7. 浏览器解析资源：解析 HTML 生成DOM Tree -> 解析 CSS 生成 CSSOM Tree -> render Tree -> layout -> compositing -> painting
8. 页面呈现

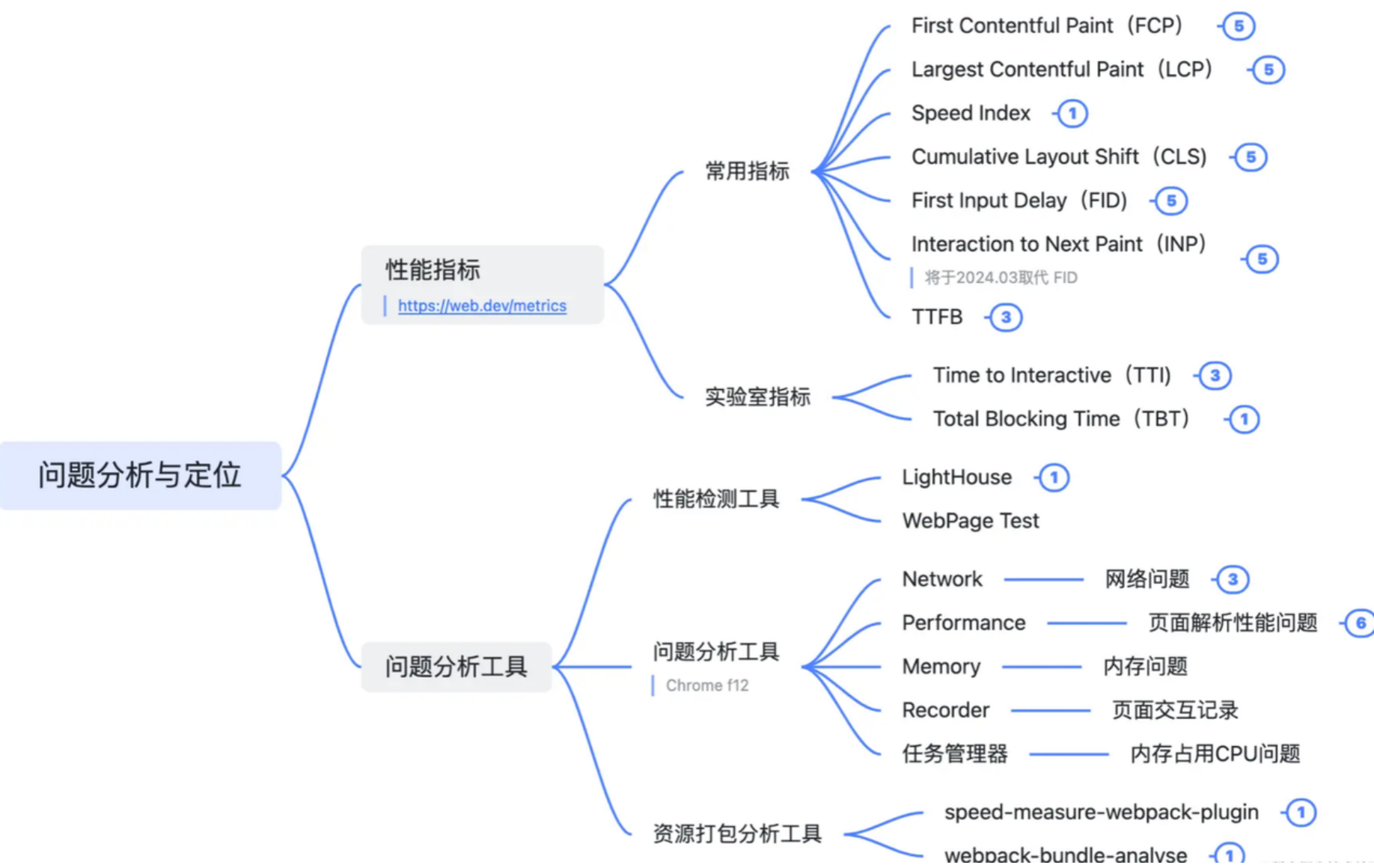
6.1 哪些因素会影响性能？

我们再从宏观角度来看，页面通常可分为3个阶段：初始化加载阶段、运行时交互阶段、关闭卸载阶段。我们从这3个阶段，同时结合上述页面初始化加载经历8大过程，可分析出影响页面性能的因素：

- **加载阶段**：从发起请求到渲染出完整的页面过程，影响这一阶段的因素有**网络**、**HTML**、**图片**、**CSS**和**JS脚本**
- **交互阶段**：从页面加载后用户交互（比如：点击、滚动页面等）的过程，影响这一阶段的因素主要是**网络**、**JS脚本**
- **关闭阶段**：用户发出关闭质量页面做一些清理卸载工作，影响这一阶段的因素主要是**JS脚本**

7. 问题分析和定位

当我们碰到页面性能、构建效率慢，我们首先需要分析定位具体问题。我们常用的参考性能指标和可视化问题分析工具主要有如下几种：



7.1 用户角度的性能指标

7.1.1 用户感知的指标

- 感知的加载速度：网页加载并将其所有视觉元素呈现到屏幕的速度
- 加载响应速度：网页加载和执行任何必需的 JavaScript 代码的速度，以便让组件快速响应用户互动
- 运行时响应速度：网页加载后，网页能够以多快的速度响应用户互动
- 视觉稳定性：网页上的元素是否会以用户意想不到且可能会干扰互动的方式发生变化，比如图片加载后出现偏移闪动
- 流畅性：过渡和动画否以一致的帧速率渲染并流畅呈现

著名 2-5-8 原则：

- 当用户能够在 2 秒以内得到响应时，会感觉系统的响应很快；
- 当用户在 2-5 秒之间得到响应时，会感觉系统的响应速度还可以；
- 当用户在 5-8 秒以内得到响应时，会感觉系统的响应速度很慢，但是还可以接受；
- 而当用户在超过 8 秒后仍然无法得到响应时，会感觉系统糟透了，或者认为系统已经失去响应，而选择离开这个 Web 站点，或者发起第二次请求。

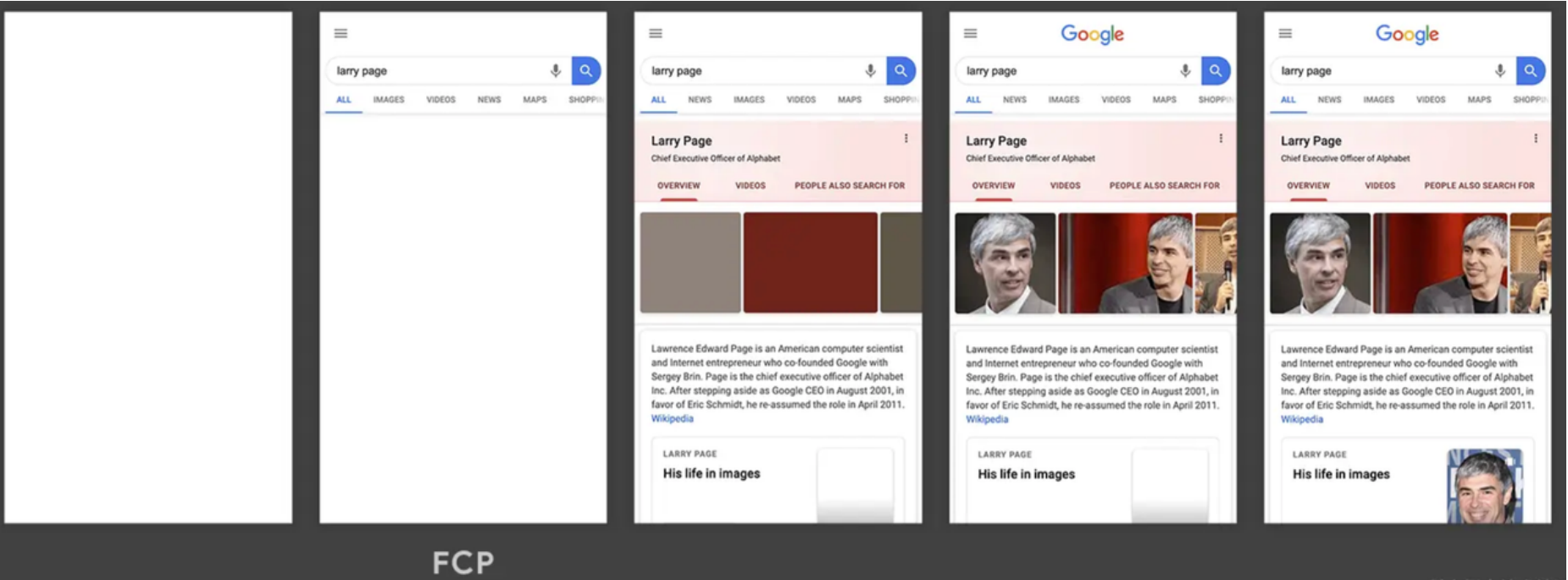
7.2 性能指标

<https://web.dev/articles/lcp?hl=zh-cn>

7.2.1 FCP（First Contentful Paint）首次内容绘制

什么是FCP？

FCP，首次内容绘制。该指标用于测量从网页加载到任何一部分呈现在屏幕上的时间。用户访问 Web 页面过程中，在 FCP 之前页面处于白屏，没有实际内容。（如下图所示）



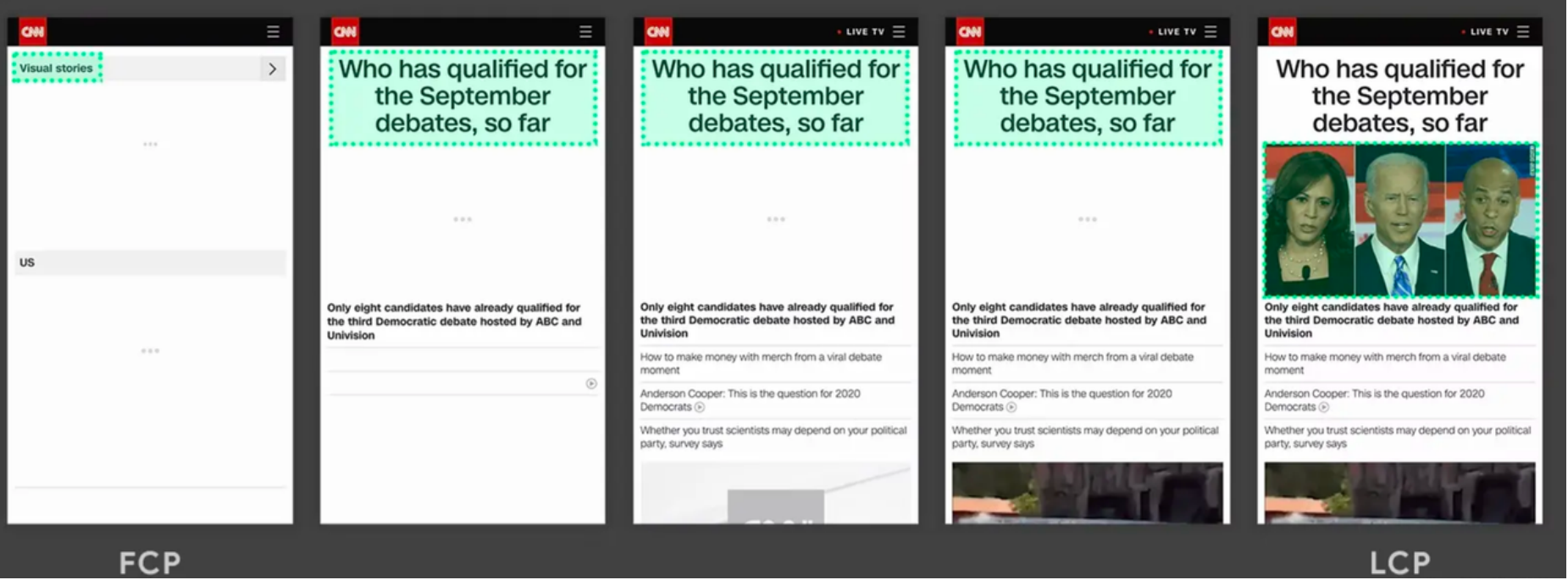
为了达到良好的用户体验，网页应尽力将 FCP 控制在1.8秒以内，不妨将网页加载的**第75百分位**作为衡量阈值。



- 小于 **1.8** 秒，表示页面加载较快，白屏时间短
- **1.8** 秒至 **3** 秒，表示页面加载较慢，但基本能接受
- **3** 秒以上，表示页面加载很慢，白屏时间过长

7.2.2 LCP（Largest Contentful Paint）最大内容绘制

LCP，最大内容绘制。该指标用来测量从网页加载到网页最大内容（文本块/图片元素）呈现在屏幕上的时间。



最大内容元素有哪些？

- `<img>` 元素
- `<svg>` 内的 `<image>` 元素

- 包含海报图片的 `<video>` 元素
- 为自动播放 `<video>` 元素而绘制的第一帧
- 动画图片（如 `GIF` 动画）的第一帧
- 包含文本节点或其他内嵌级别文本元素的子项的块级元素

为了达到良好的用户体验，网页应尽力将 LCP 控制在2.5秒以内，不妨将网页加载的**第75百分位**作为衡量阈值。

LCP

Largest Contentful Paint



7.2.3 FID（First Input Delay）

FID，测量的是用户首次与网页互动（点链接、点按钮等）到浏览器响应该互动的的时间。

7.2.4 TTFB 首字节时间

TTFB，用于测量资源请求与响应第一个字节之间的时间。

TTFB

Time To First Byte



7.2.5 TTI（Time To Interactive）可交互时间

TTI，用于测量从网页开始加载到主要子资源已加载且能够快速响应用户输入的时间。（TTI，即是静默期之前的最后一个长任务结束时间。如果未找到长任务，则与FCP相同）

7.3 性能指标的计算

7.3.1 引入 web-vitals 官方库进行计算

Code block

```
1  import { onFCP, onLCP, onFID, TTFB } from 'web-vitals';
2
3  onFCP(console.log);
4  onLCP(console.log);
5  onFID(console.log);
6  onTTFB(console.log);
```

7.3.2 通过performance api 进行计算

通过 `performance api`，不仅可以计算网络相关耗时，也可以计算性能指标，比如白屏时间FP、FCP、LCP、FID、CLS、TTFB等。

Performance.timing 被弃用后，可通过 [PerformanceObserver](#) 来获取性能指标，具体计算如下：

使用 performance.getEntries 等方法获取性能信息的前提是对应的 Performance Entry 已经产生，如对应的资源加载已经完成、页面已经完成渲染等，在加载资源前是获取不到任何其加载的性能信息的。

为了能够判断相应的时机，可以用浏览器提供的 PerformanceObserver 来监听Performance Entry 相关的事件。

白屏时间 FP

白屏时间 FP（First Paint）指的是从用户输入url开始到页面有内容展现出来的时间节点。（这个过程包括DNS查询、TCP连接、HTTP请求、返回 HTML、解析HTML）

FP 通过 `PerformanceObserver` 实例对象中监听名为 `first-paint` 的 `paint` 条目获取，具体代码如下：

Code block

```
1  const entryHandler = (list) => {
2    for(const entry of list.getEntries()){
3      if(entry.name === 'first-paint') {
4        observer.disconnect()
5      }
6      // 白屏时间
7      let FP = entry.startTime;
8    }
9  }
10 const observer = new PerformanceObserver(entryHandler)
11 observer.observe({type: 'paint', buffered: true})
```

首次内容绘制 FCP

FCP 通过 `PerformanceObserver` 实例对象中监听名 `first-contentful-paint` 的 `paint` 条目获取，具体代码如下：

Code block

```
1  const entryHandler = (list) => {
2    for(const entry of list.getEntries()){
3      if(entry.name === 'first-contentful-paint') {
4        observer.disconnect()
5      }
6      // 首次内容绘制时间
7      let FCP = entry.startTime;
8    }
9  }
10 const observer = new PerformanceObserver(entryHandler)
11 observer.observe({type: 'paint', buffered: true})
```

最大内容绘制 LCP

LCP 通过 `PerformanceObserver` 实例对象的 `largest-contentful-paint` 条目获取，具体代码如下：

Code block

```
1  const entryHandler = (list) => {
2    for(const entry of list.getEntries()){
3      observer.disconnect()
4      // 最大内容绘制
5      let LCP = entry.startTime
6    }
7  }
```

```
7 }
8 const observer = new PerformanceObserver(entryHandler);
9 observer.observe({ type: 'largest-contentful-paint', buffered: true });
```

7.3.3 性能监控实现

Code block

```
1  /**
2   * 性能监控类，用于收集和上报页面性能指标
3   */
4  class PerformanceMonitor {
5      // 存储性能指标的容器
6      private metrics: Record<string, any> = {}
7      // 需要监控的资源类型
8      private readonly RESOURCE_TYPES = ['img', 'css', 'script']
9      // 性能数据上报的API端点
10     private readonly PERFORMANCE_ENDPOINT = '/performance'
11
12     constructor() {
13         this.init()
14     }
15
16     /**
17      * 初始化性能监控
18      */
19     private init(): void {
20         this.setupLoadPerformanceMonitoring()
21         this.setupResourcePerformanceMonitoring()
22     }
23
24     /**
25      * 设置页面加载性能监控
26      * 主要监控首次内容绘制时间(FCP)
27      */
28     private setupLoadPerformanceMonitoring(): void {
29         const reportPerformance = () => {
30             const paint = performance.getEntriesByType('paint')
31             const fcpEntry = paint.find(
32                 (entry) => entry.name === 'first-contentful-paint'
33             )
34
35             if (fcpEntry) {
36                 this.metrics.FCP = fcpEntry.startTime
37                 this.reportMetrics()
38             }
39         }
40
41         window.addEventListener('load', () => {
42             setTimeout(reportPerformance, 0)
43         })
44     }
45
46     /**
47      * 设置资源加载性能监控
48      * 使用PerformanceObserver监控指定类型的资源加载时间
49      */
50     private setupResourcePerformanceMonitoring(): void {
51         const observer = new PerformanceObserver((list) => {
52             list.getEntries().forEach((entry) => {
53                 if (this.RESOURCE_TYPES.includes(entry?.initiatorType)) {
```



```

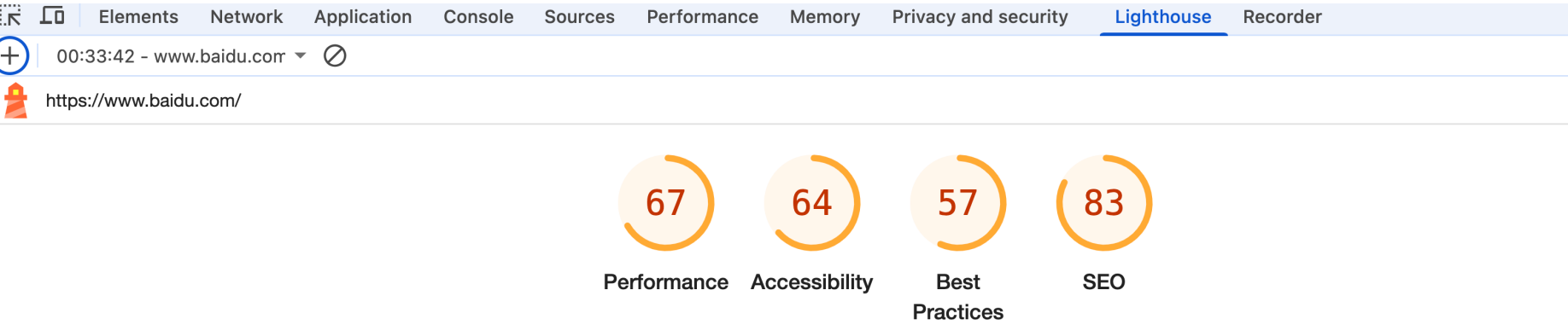
54         this.metrics[entry.name] = entry.duration
55     }
56 })
57 })
58
59     observer.observe({ entryTypes: ['resource'] })
60 }
61
62 /**
63  * 上报性能指标
64  * 使用navigator.sendBeacon方法异步上报数据
65  */
66 private reportMetrics(): void {
67     console.log('JSON.stringify(this.metrics)', JSON.stringify(this.metrics))
68     try {
69         navigator.sendBeacon(
70             this.PERFORMANCE_ENDPOINT,
71             JSON.stringify(this.metrics)
72         )
73     } catch (error) {
74         console.error('性能指标上报失败:', error)
75     }
76 }
77 }
78
79 // 初始化性能监控
80 new PerformanceMonitor()
81
82 // app.vue
83 import { useEffect, useState } from 'react'
84 import { onLCP, onTTFB, onFCP } from 'web-vitals'
85
86 import './utils/performance'
87 import './App.css'
88
89 const vDom = <h1>hello</h1>

```

7.4 性能检测工具

7.4.1 Lighthouse

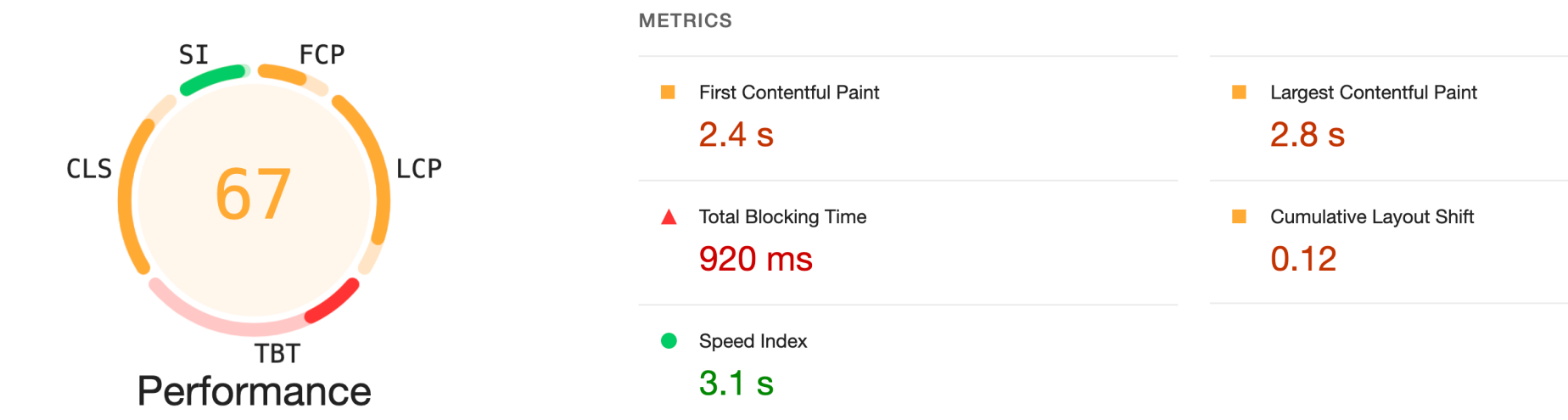
Chrome开发者工具Lighthouse面板，主要针对“Performance性能、Accessibility可访问性、Best Practice最佳实践、SEO搜索引擎优化、Progressive Web App离线应用” 5个方面做了分析。我们常使用该工具做页面Performance性能检测，并制定合适优化方案落地。



There were issues affecting this run of Lighthouse:

- Chrome extensions negatively affected this page's load performance. Try auditing the page in incognito mode or from a Chrome profile without extensions.

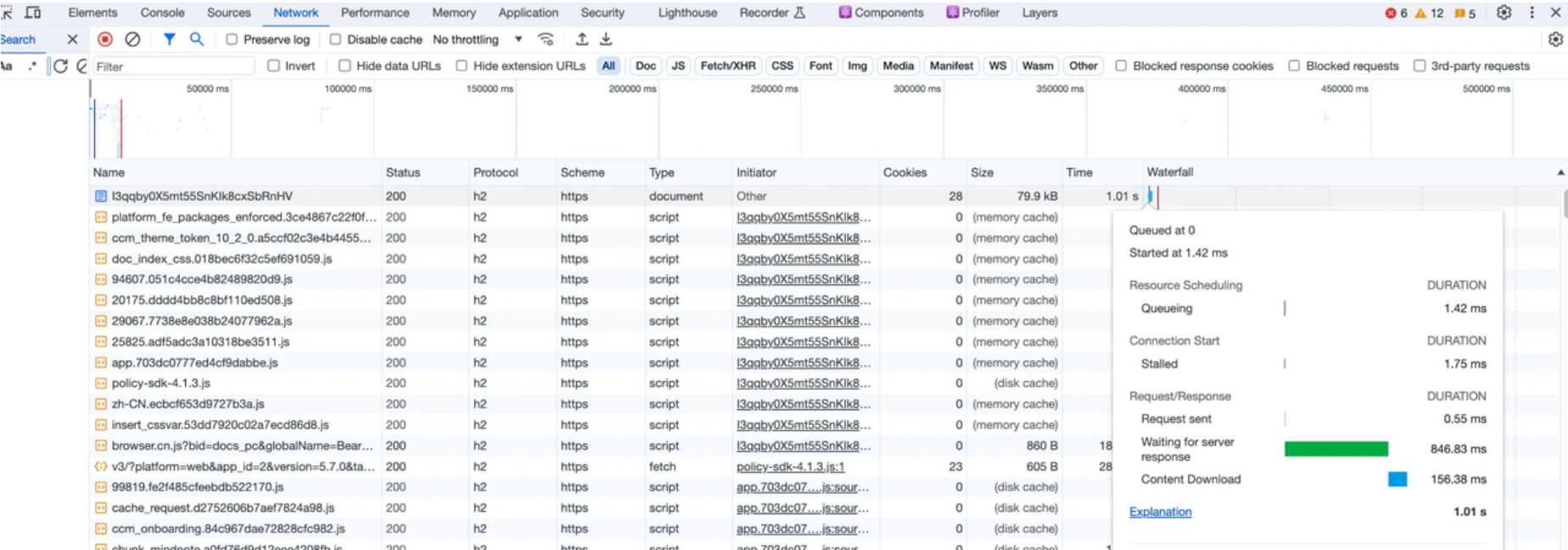
Performance 性能区域列出了 FCP、LCP、TBT、CLS、Speed Index 5个核心指标，并给出具体优化方案。



7.5 问题分析定位工具

7.5.1 Network 面板

Network 面板主要是用来分析定位网络请求问题，其面板主要有“过滤、概览、查看详情”3个功能区域。

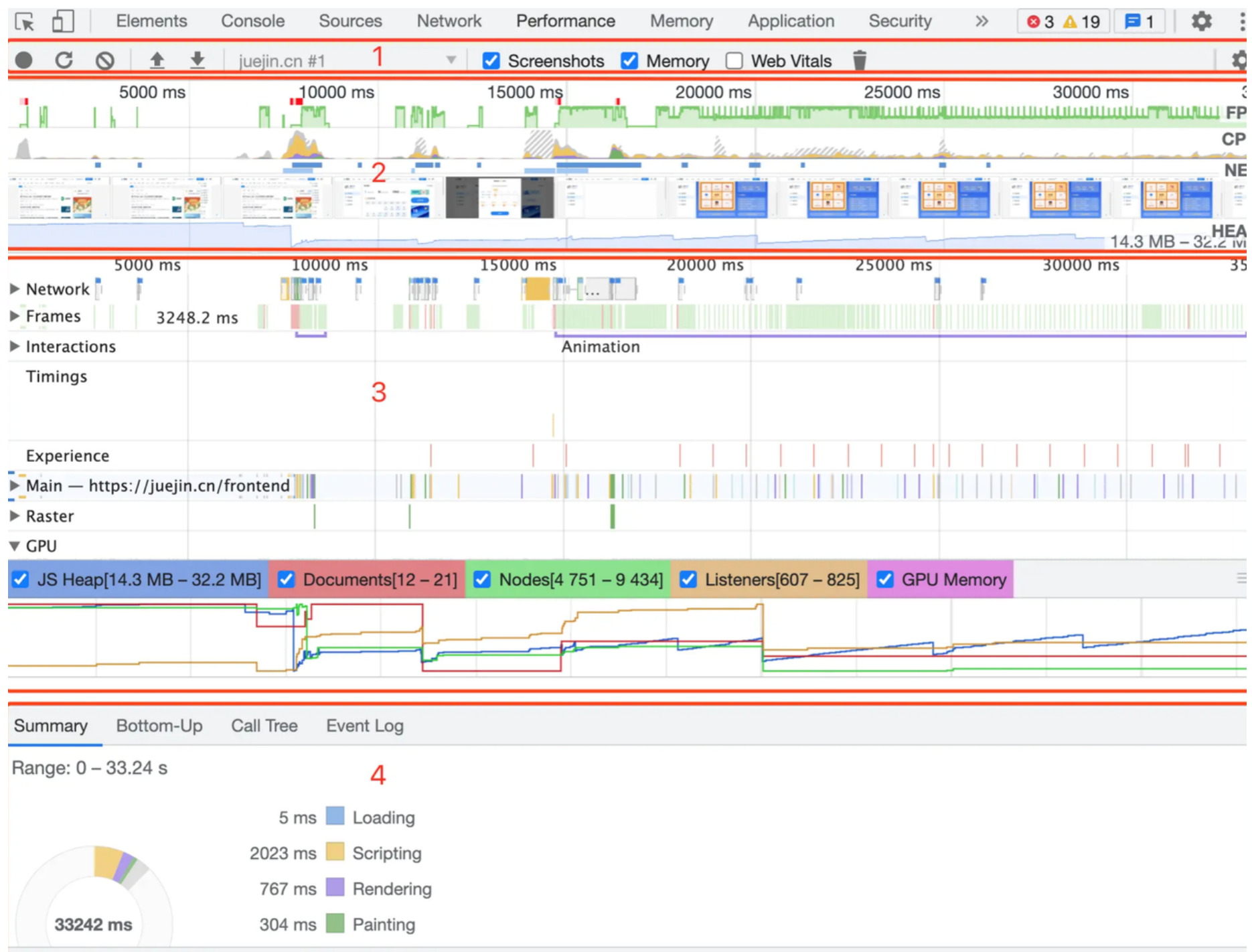


Network面板可查看的基本信息有：

- 请求的name名称、size大小、method方法、status状态码、scheme协议类型、protocol协议版本、type类型、initiator调用堆栈等
- 请求的响应时长
- 请求的瀑布图waterfall
 - nQueued at 浏览器将资源放入队列时间
 - Started at 浏览器开始处理资源的时间
 - Queueing 资源放入队列的等待处理时间（如果遇到高优先级或请求超过并发限制，需要等待）
 - Stalled 因放入队列而发生的停滞时间
 - Request sent 请求发送的时间
 - Waiting for server response 等待服务器端返回数据的时间（TTFB）
 - Content Download 浏览器下载资源的时间

7.5.2 Performance面板（定位页面渲染性能问题）

Perfomance 面板，主要分析定位页面渲染性能问题（HTML解析、JS执行、CSS解析布局计算重排、Painting绘制）。我们来看看该面板几个区域的主要功能：



区域1：控制面板：录制页面加载/操作、清空、快照/内存捕捉等

区域2：概览面板

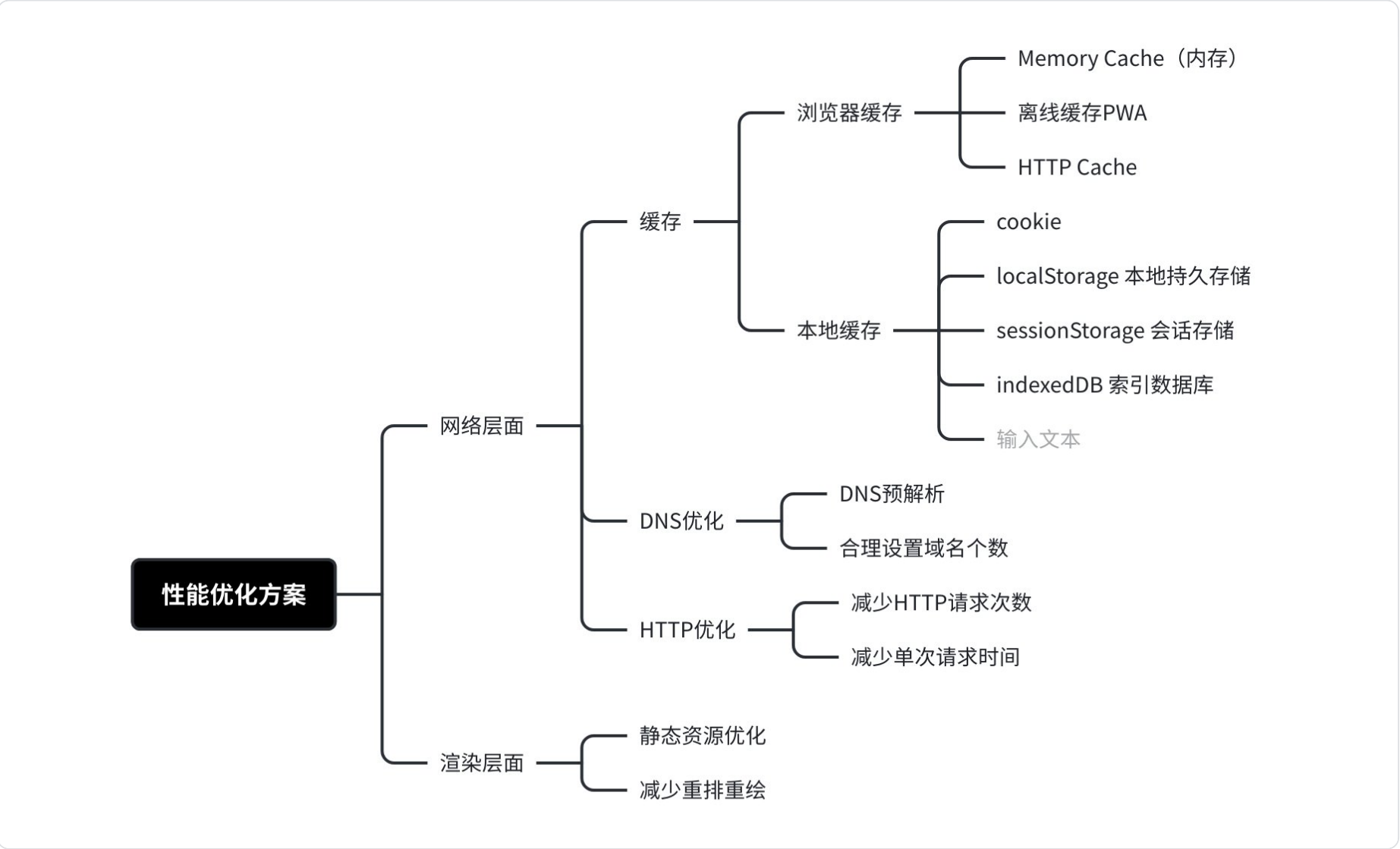
- FPS：帧率，每秒帧数
- CPU：处理各个任务花费的时间
- NET：各个请求花费时间
- HEAP：内存消耗记录，包括存在内存未销毁的js变量、文档量、dom节点、事件监听、GPU占用内存等

区域3：线程面板

- Network：各个请求发生的时间点
- Frames：帧线程
- Main：主线程，负责加载、执行JavaScript，解析html、css，生成render树，完成绘制painting（帮助我们理解浏览器的页面渲染原理）
 - 主线程活动。通过倒置的火焰图展示主线程上发生的活动，x轴表示随时间的记录。y轴代表调用堆栈。上层的事件调用（触发）了下层的事件（anonymous代表匿名函数），火焰图顶层宽度越大就表示该活动可能存在性能问题。
 - 查看长任务
 - 火焰图顶部（根部）由很多任务（Task）组成，使用灰色背景色区分。鼠标悬浮上去可以看到任务的总耗时。

- Painting: 重绘。更新分层、光栅化分层、合成等。
 - System: 系统占用时间。
 - Idle: 空闲时间。
 - Total: 总计时间
- 分类中的详细事件可[点击这里](#)

前面使用分析工具定位问题后，我们就可以制定相应的优化方案，可以从“网络层面、渲染层面”两方面展开探索~



8. 网络层面优化

8.1 使用缓存

使用缓存可以减少网络 IO 消耗，大幅提高访问速度。但开发在“网络层面：缓存命中”可做的并不多，能介入优化的主要是“浏览器缓存”这一阶段，下面我们具体来看看优化方案有哪些？

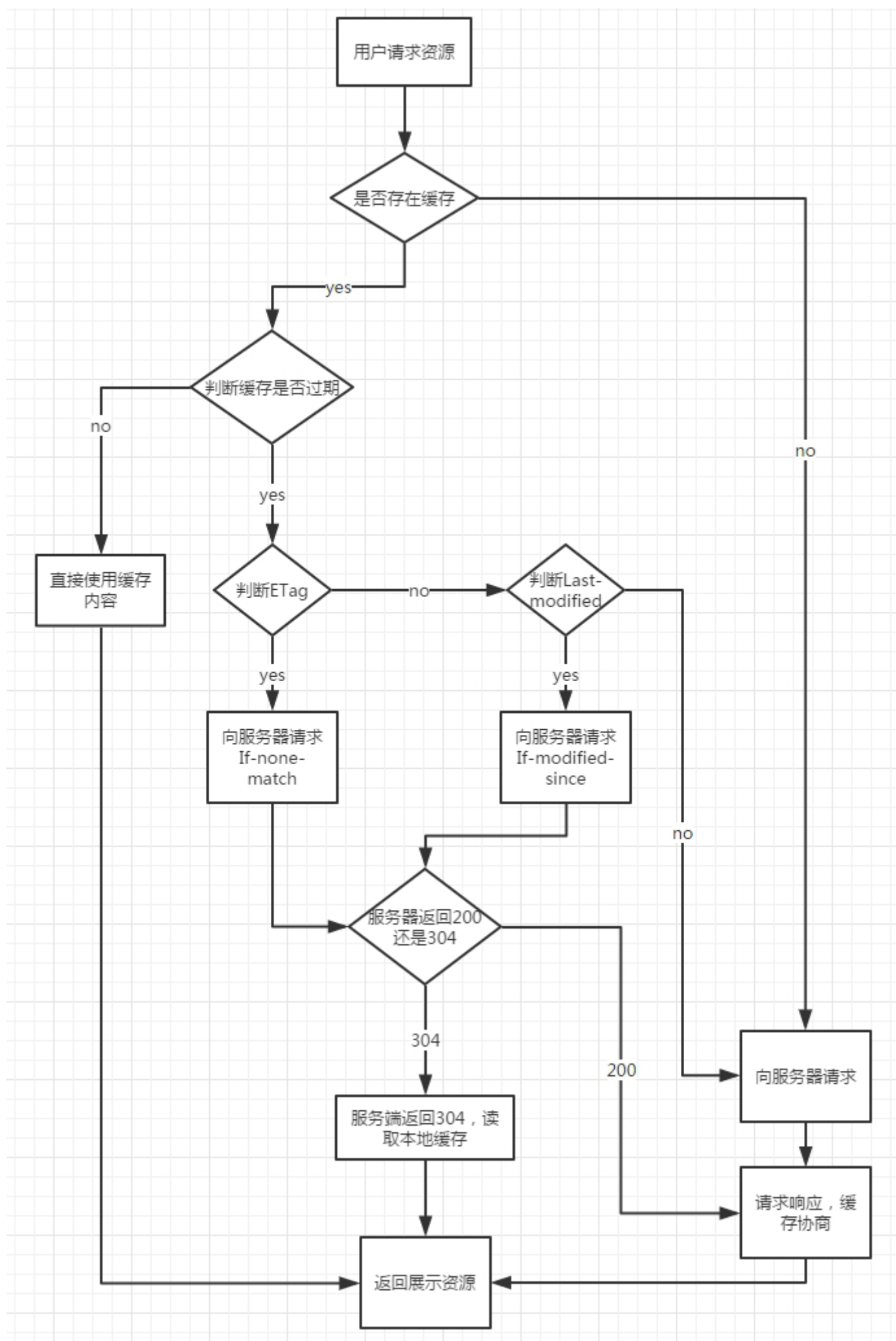
8.2 浏览器缓存

大多时候，大家将“浏览器缓存”简单地理解成“HTTP缓存”，实际上它分为4个方面，按获取资源的请求优先级排列如下：

1. Memory Cache（内存缓存）
2. Service Worker Cache（离线缓存）
3. HTTP Cache（HTTP 缓存）
4. Push Cache（HTTP2 推送缓存）

Push Cache是HTTP2的新特性，升级HTTP2版本即可，我们接下来更多还是从“Memory Cache、离线缓存、HTTP缓存”3个方面来看如何使用浏览器缓存，进而提升页面性能。

8.2.1 HTTP缓存



8.2.1.1 状态码

- 200：强缓存 Expires / Cache-Control 失效时，返回新资源文件
- 200（from disk cache）Expires / Cache-Control 两者都存在且有效，Cache-Control 优先 Expires 时，浏览器从本地获取资源成功。
- 200（from memory cache）
- 304（Not Modified）协商缓存 Last-modified / Etag 有效，则服务端返回该状态码。

Cache-Control

- **no-cache** 存储在本地缓存取中，只是在与服务器进行新鲜度再验证之前，缓存无法使用。

- **no-store** 不缓存资源到本地
- **public** 可被所有用户缓存，多用户进行共享，包括终端或 CDN 等中间代理服务器
- **private** 仅能被浏览器客户端缓存，属于私有缓存，不允许中间代理服务器缓存相关资源

8.2.1.2 内存还是磁盘

通过 `Cache-Control` 头部间接引导缓存位置

1. 高频访问的小资源 → 倾向于内存缓存

Code block

```
1 Cache-Control: max-age=300, must-revalidate
```

2. 低频访问的大资源 → 倾向于磁盘缓存

Code block

```
1 Cache-Control: public, max-age=31536000, immutable
```

8.2.1.3 如何缓存

缓存启用的顺序可列举如下：

1. Cache-Control —— 请求服务器之前
2. Expires —— 请求服务器之前
3. If-None-Match (Etag) —— 请求服务器
4. If-Modified-Since (Last-Modified) —— 请求服务器

需要注意的是协商缓存需要配合强缓存使用，如果不启用强缓存那么协商缓存就失去了意义。大部分 web 服务器都默认开启了协商缓存，而且是同时启用（Last-Modified、If-Modified-Since）和（ETag、If-None-Match）。但当我们的系统选用分布式部署时，则需要注意以下问题：

- 分布式系统里多台机器间文件的 Last-Modified 必须保持完全一致，否则在请求负载均衡到不同机器时，会导致比对失败的情况；
- 分布式系统尽量关闭掉 ETag，因为每台机器生成的 ETag 都不同。

Code block

```
1 const http = require('http')
2 const fs = require('fs')
3 const url = require('url')
4 const path = require('path')
5 const etag = require('etag')
6 const fresh = require('fresh')
7
8 const server = http.createServer(function (req, res) {
9   let filePath, isHtml, isFresh
10   // true 这个布尔值参数表示对查询字符串进行解析并将其转换为一个对象
11   // pathname: URL 的路径部分
12   const pathname = url.parse(req.url, true).pathname
13
14   console.log('pathname: ' + pathname)
15   //根据请求路径取文件绝对路径
16   if (pathname === '/') {
17     filePath = path.join(__dirname, 'public', '/index.html')
```

```

18     isHtml = true
19   } else {
20     filePath = path.join(__dirname, 'public', pathname)
21     isHtml = false
22   }
23
24   // 读取文件描述信息，用于计算etag及设置Last-Modified
25   // fs.stat()可以返回关于文件的各种信息，例如文件大小、创建时间、修改时间等。
26   fs.stat(filePath, function (err, stat) {
27     console.log('stat', stat)
28     if (err) {
29       res.writeHead(404, 'not found')
30       res.end('<h1>404 Not Found</h1>')
31     } else {
32       if (isHtml) {
33         // html文件使用协商缓存
34         const lastModified = stat.mtime.toUTCString() // mtime文件最近一次修改的时间戳
35         const fileEtag = etag(stat)
36         res.setHeader('Cache-Control', 'public, max-age=0')
37         res.setHeader('Last-Modified', lastModified)
38         res.setHeader('ETag', fileEtag)
39
40         // 根据请求头判断缓存是否是最新的
41         isFresh = fresh(req.headers, {
42           etag: fileEtag,
43           'last-modified': lastModified,
44         })
45       } else {
46         // 其他静态资源使用强缓存
47         res.setHeader('Cache-Control', 'public, max-age=3600')
48       }
49
50       fs.readFile(filePath, 'utf-8', function (err, fileContent) {
51         if (err) {
52           res.writeHead(404, 'not found')
53           res.end('<h1>404 Not Found</h1>')
54         } else {
55           if (isHtml && isFresh) {
56             //如果缓存是最新的 则返回304状态码
57             //由于其他资源使用了强缓存 所以不会出现304
58             res.writeHead(304, 'Not Modified')
59           } else {
60             res.write(fileContent, 'utf-8')
61           }
62
63           res.end()
64         }
65       })
66     }
67   })
68 })
69 server.listen(3000, () => {
70   console.log('server is running on 3000')
71 })

```

8.2.2 service worker

Service Worker 本质上充当 Web 应用程序、浏览器与网络（可用时）之间的代理服务器，可以拦截资源请求根据自己的逻辑做处理，比如可以拦截网络请求、缓存资源、离线访问等。

- Service Worker 注册

- a. 获取 service worker 代码，然后使用 `serviceWorkerContainer.register()` 来注册。如果成功，service worker 将在 `ServiceWorkerGlobalScope` 中执行；这本质上是一种特殊的上下文，在主脚本执行线程之外运行，没有访问 DOM 的权限。Service Worker 现在已为处理事件做好准备。

Code block

```
1  if ('serviceWorker' in navigator) {
2    navigator.serviceWorker.register('/service-worker.js', {
3      scope: '/'
4    }).then(function (registration) {
5      // 注册成功
6      console.log('ServiceWorker registration successful with scope: ', registration.scope);
7    }).catch(function (err) {
8      // 注册失败 :(
9      console.log('ServiceWorker registration failed: ', err);
10   });
11 }
12
```

- Service Worker 生命周期
 - 安装 install

Code block

```
1  self.addEventListener('install', function (event) {
2    event.waitUntil(
3      // 这里可以做一些缓存的操作
4      // 考虑到缓存也需要更新，open内传入的参数为缓存的版本号
5      caches.open('test-v1').then((cache) => {
6        return cache.addAll([
7          // 此处传入指定的需缓存的文件名
8          '/test.html',
9          '/test.css',
10         '/test.js',
11       ]);
12     });
13   });
14 });
15
```

- i. 当 `install` 程序处理完成时，service worker 被视为已安装。此时，service worker 的先前版本可能处于激活的状态并控制着打开的页面。由于我们不希望同一 service worker 的两个不同版本同时运行，因此新版本尚未激活。

- 激活 activate
 - 一旦 service worker 的旧版本控制的页面都已关闭，就可以安全地停用旧版本，并且新安装的 service worker 将收到 `activate` 事件。`activate` 的主要用途是去清理 service worker 之前版本使用的资源。新的 service worker 可以调用 `skipWaiting()` 要求立即激活，而无需要求打开的页面关闭。然后，新的 service worker 将立即收到 `activate` 事件，并将接管任何打开的页面。

Code block

```
1  self.addEventListener('activate', function (event) {
2    event.waitUntil(
3      // 这里可以做一些清理缓存的操作
4    );
5  });
6
7
```

- 运行

Code block

```
1 self.addEventListener('fetch', function (event) {
2   // 拦截请求
3   // 策略：先从缓存中获取，若缓存中没有发起请求从网络中获取
4   event.respondWith(
5     caches.match(event.request).then(function (response) {
6       if (response) {
7         return response;
8       }
9       return fetch(event.request);
10    })
11  );
12 });
```

8.3 资源压缩

Code block

```
1 // webpack.config.js
2 const TerserPlugin = require('terser-webpack-plugin');
3 const CssMinimizerPlugin = require('css-minimizer-webpack-plugin');
4
5 module.exports = {
6   optimization: {
7     minimizer: [
8       new TerserPlugin({
9         terserOptions: {
10           compress: {
11             drop_console: true,
12           },
13         },
14       }),
15       new CssMinimizerPlugin(),
16     ],
17   },
18 };
```

8.4 DNS优化

在 HTML 中优化 DNS 解析主要通过 **减少 DNS 查询次数** 和 **提前解析关键域名** 来提升页面加载速度。以下是具体实现方法和最佳实践：

8.4.1 DNS 预取（DNS Prefetching）

通过 HTML 的 `<link>` 标签提示浏览器提前解析关键域名的 DNS，减少后续资源请求的延迟。

Code block

```
1 <!-- 预解析关键域名 -->
2 <link rel="dns-prefetch" href="//cdn.example.com">
3 <link rel="dns-prefetch" href="//api.example.com">
4 <link rel="dns-prefetch" href="//fonts.googleapis.com">
```

适用场景：

- 页面中引用了第三方资源（如 CDN、字体、分析脚本）。

注意事项：

- 避免过度预解析，通常建议预解析 **3-5 个关键域名**。
- 现代浏览器（Chrome、Firefox、Edge）默认支持 `dns-prefetch`。

8.4.2 预连接（Preconnect）

在 DNS 预取的基础上，进一步建立 TCP 连接和 TLS 握手，适用于高频访问的域名。

Code block

```
1  <!-- 提前建立连接 (DNS + TCP + TLS) -->
2  <link rel="preconnect" href="https://cdn.example.com" crossorigin>
```

适用场景：

- 高频请求的核心 API 域名。
- 静态资源 CDN 域名。

8.4.3 域名收敛

域名收敛（Domain Sharding 或 Domain Consolidation）是指将网站的资源（如静态文件、API 等）从多个域名合并到少数几个甚至单个域名的策略。这一策略在 **HTTP/2 普及后** 逐渐被重视，其核心目标是通过减少域名数量优化网站性能、简化管理并提升安全性。以下是域名收敛的主要好处。

优化前（多域名）：

Code block

```
1  <script src="https://static1.example.com/app.js"></script>
2  <link href="https://static2.example.com/style.css" rel="stylesheet">
3  
```

优化后（单域名 + CDN）：

Code block

```
1  <script src="https://cdn.example.com/app.js"></script>
2  <link href="https://cdn.example.com/style.css" rel="stylesheet">
3  
```

优势：

- 减少 DNS 查询次数。
- 利用 HTTP/2 多路复用特性，避免重复 TCP 握手。

8.4.3.1 减少 DNS 解析开销

- 问题：每个新域名都需要独立的 DNS 解析，增加延迟（尤其移动端网络）。
- 收敛好处：
 - 减少域名数量 → 减少 DNS 查询次数。
 - 缩短整体解析时间，加速页面首次加载（尤其是低延迟网络环境）。

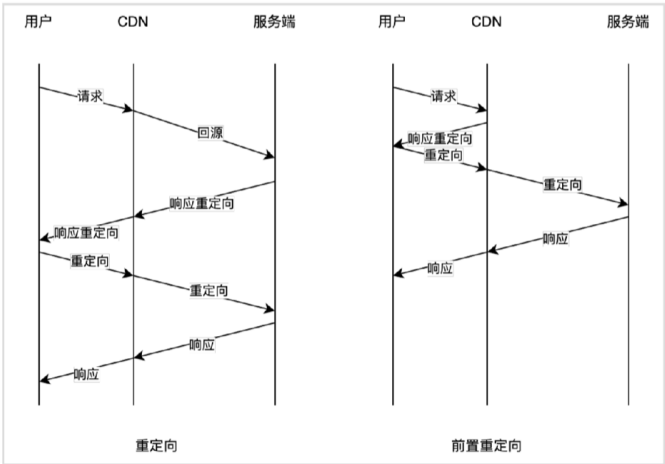
8.4.3.2 避免不必要的 TCP 连接和 TLS 握手

- HTTP/1.1 的并发限制：
浏览器对同一域名的并发请求数有限（如 Chrome 默认 6 个），过去常通过多域名（如 `static1.example.com`，`static2.example.com`）突破限制。
- HTTP/2 多路复用：
HTTP/2 支持单连接多路复用，无需多域名即可并发传输资源。
- 收敛好处：
 - 减少 TCP 连接数 → 降低连接建立和 TLS 握手开销。
 - 避免因过多域名导致的连接竞争和资源浪费。

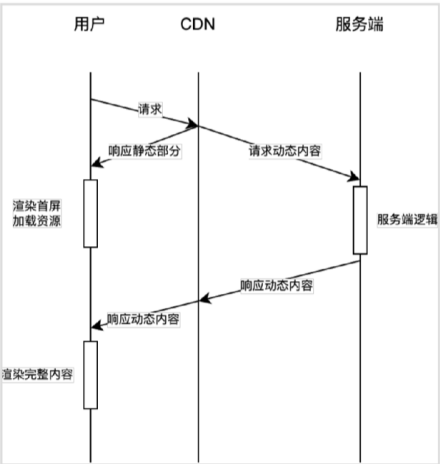
8.4.3.3 适用场景与权衡

- 推荐收敛的场景：
 - 使用 HTTP/2 或 HTTP/3 的现代网站。
 - 移动端或高延迟网络环境（需减少 DNS 和连接开销）。
 - 需简化运维和安全管理的企业级应用。
- 需谨慎的场景：
 - 仍需支持 HTTP/1.1 的旧系统（可适度保留少量域名）。
 - 第三方资源（如广告、分析脚本）需隔离时。

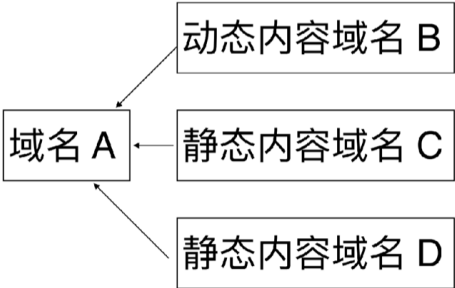
8.5 CDN



前置重定向



流式渲染



域名收敛

9. 渲染层面优化

- 尽可能减少资源个数
- 尽可能减少资源体积大小

9.1 减少文件大小（压缩、精简）

- 压缩处理 HTML，减小HTML体积
- 精简 HTML：
 - 尽量减少 HMTL 嵌套、iframe/table 使用（table 标签比其他html标签占用更多字节，导致下载时间长，占用服务器更多的流量资源）

- 删除多余的空格、换行符、缩进和不必要注释
- 删除冗余标签和属性

9.2 DOM优化

- 控制DOM大小：
 - 合理的业务逻辑拆分
 - 先加载可视区，其他延迟加载（懒加载）
- 减少DOM操作：尽可能对DOM操作统一逻辑处理，或是使用虚拟DOM（借鉴vue/react），再插入到真实DOM（减少重排重绘）

9.3 CSS优化

9.3.1 减少资源请求个数

- 合并多个 CSS 样式文件
- 按需加载样式

9.3.2 减少文件大小

- 压缩处理 CSS 文件
- 位置放在 `<head>` 里，尽早地进行样式解析，构建CSSOM 树
- 简化 CSS 选择器（选择器优先级：`!important` > 内联 > `id` > `class`|属性|伪类 > 标签 | 伪元素）
 - 尽可能减少样式层级数（选择器嵌套）
 - 少用标签选择器，尽量选择高优先级的`id/class/属性/伪类`选择器代替
 - 少用通配符`*`，只对需要修改样式的元素进行选择
 - 关注可继承属性，避免重复定义和匹配

9.4 图片加载优化

9.4.1 懒加载

图片列表一般采用懒加载进行按需加载，滚屏时当图片已将出现在可视区域的时候进行加载。（有效地减轻服务器批量加载图片的压力）

思路

- 添加自定义属性：给标签添加 `data-src`，值为图片的url，同时不要设置src属性
- 判断目标元素与视口的交叉状态：通过获取元素的`getBoundingClientRect`属性的top值和页面的`clientHeight`进行对比，如果top值小于`clientHeight`，则说明元素出现在可视区域之内
- 设置真实的src：当元素出现在可视区域内时，将真实的图片地址赋值给目标元素的src属性

IntersectionObserver

IntersectionObserver 接口 (从属于Intersection Observer API) 提供了一种异步观察目标元素与其祖先元素或顶级文档视窗 (viewport) 交叉状态的方法。祖先元素与视窗 (viewport) 被称为**根 (root)。

性能优化策略

- 单例观察器
对同类元素使用同一观察器，减少内存开销：

```
1  const observer = new IntersectionObserver(callback);
2  document.querySelectorAll('.lazy').forEach(el => observer.observe(el));
```

- **动态解除观察**

触发后立即 `unobserve` 避免重复检测：

Code block

```
1  if (entry.isIntersecting) {
2    doSomething(entry.target);
3    observer.unobserve(entry.target);
4  }
```

- **合理设置阈值**

根据场景选择最佳触发时机：

- **懒加载**： `threshold: 0.01` + `rootMargin: '200px'`
- **精准曝光统计**： `threshold: 0.8`

- **批量处理元素**

对长列表分批次观察，避免同时观察数千个元素：

Code block

```
1  const chunkSize = 50;
2  const elements = [...document.querySelectorAll('.list-item')];
3  elements.forEach((el, index) => {
4    if (index % chunkSize === 0) {
5      setTimeout(() => observeChunk(elements.slice(index, index + chunkSize)), 0);
6    }
7  });
```

自定义hook

Code block

```
1  import { useEffect, useRef } from 'react';
2
3  export const useIntersectionObserver = (
4    callback: (entry: IntersectionObserverEntry) => void,
5    options?: IntersectionObserverInit
6  ) => {
7    const ref = useRef<HTMLElement>(null);
8
9    useEffect(() => {
10      const element = ref.current;
11      if (!element) return;
12
13      const observer = new IntersectionObserver([entry]) => {
14        if (entry) callback(entry);
15      }, options);
16
17      observer.observe(element);
18      return () => observer.disconnect();
19    }, [callback, options]);
20
21    return ref;
22  };
```

Code block

```
1  const observer = new IntersectionObserver((entries) => {
2    entries.forEach(entry => {
3      if (entry.isIntersecting) {
4        // 元素进入视口时执行操作
5        console.log('元素可见:', entry.target);
6        observer.unobserve(entry.target); // 仅触发一次
7      }
8    });
9  }, {
10    root: null,           // 默认视口
11    rootMargin: '200px', // 提前触发检测
12    threshold: 0.1       // 可见比例达到10%触发
13  });
14
15  // 观察元素
16  const target = document.querySelector('.lazy-element');
17  observer.observe(target);
18
19  // 具体使用
20  const observer = new IntersectionObserver((entries) => {
21    entries.forEach(entry => {
22      if (entry.isIntersecting) {
23        const img = entry.target;
24        img.src = img.dataset.src;
25        observer.unobserve(img);
26      }
27    });
28  }, { rootMargin: '200px' }); // 提前200px触发加载
29
30  document.querySelectorAll('img[data-src]').forEach(img => observer.observe(img));
```

Code block

```
1  // LazyImage.tsx
2  import { useEffect, useRef, useState } from 'react';
3
4  interface LazyImageProps {
5    src: string;
6    alt: string;
7    srcSet?: string;
8    sizes?: string;
9    width?: number;
10   height?: number;
11   className?: string;
12   threshold?: number;
13   rootMargin?: string;
14 }
15
16  const LazyImage = ({
17    src,
18    alt,
19    srcSet,
20    sizes,
21    width,
22    height,
```

```
23   className,
24   threshold = 0.01,
25   rootMargin = '200px'
26 }: LazyImageProps) => {
27   const imgRef = useRef<HTMLImageElement>(null);
28   const [isLoading, setIsLoaded] = useState(false);
29   const [isInView, setIsInView] = useState(false);
30   const [hasError, setHasError] = useState(false);
31
32   // 加载真实图片
33   useEffect(() => {
34     if (!isInView || !imgRef.current) return;
35
36     const img = new Image();
37     img.src = src;
38     if (srcSet) img.srcset = srcSet;
39     if (sizes) img.sizes = sizes;
40
41     img.onload = () => {
42       if (imgRef.current) {
43         imgRef.current.src = src;
44         if (srcSet) imgRef.current.srcset = srcSet;
45         if (sizes) imgRef.current.sizes = sizes;
46         setIsLoaded(true);
47       }
48     };
49
50     img.onerror = () => {
51       setHasError(true);
52     };
53   }, [isInView, src, srcSet, sizes]);
54
55   // 初始化 IntersectionObserver
56   useEffect(() => {
57     const observer = new IntersectionObserver(
58       ([entry]) => {
59         if (entry.isIntersecting) {
60           setIsInView(true);
61           observer.disconnect();
62         }
63       },
64       {
65         threshold,
66         rootMargin,
67         root: null
68       }
69     );
70
71     if (imgRef.current) {
72       observer.observe(imgRef.current);
73     }
74
75     return () => {
76       if (imgRef.current) {
77         observer.unobserve(imgRef.current);
78       }
79     };
80   }, [threshold, rootMargin]);
81
82   return (
83     <div
84       style={{
```

```

85         position: 'relative',
86         width: width ? `${width}px` : '100%',
87         height: height ? `${height}px` : 'auto'
88     }}
89     className={className}

```

- 预加载

预加载preload，在大图片加载完成前先加载小的loading，用于提升用户体验。（该优化思想不仅可以用于图片加载，也能用于异步请求、html标签预加载）

Code block

```

1  // 创建 img 图片元素
2  const myImage = (function(){
3      let imgNode = document.createElement('img')
4      document.body.appendChild( imgNode )
5      return {
6          setSrc: function(src) {
7              imgNode.src = src
8          }
9      }
10 })()
11
12 /**
13  * 预加载
14  */
15 const preLoad = (function(){
16     let img = new Image();
17     img.onload = function() {
18         myImage.setSrc( this.src )// this指向img
19     }
20     return {
21         setImg: function(src) {
22             myImage.setSrc('./img/loading.gif')
23             img.src = src
24         }
25     }
26 })()
27
28 preLoad.setImg('./img/bg_gaoqing.jpeg')
29

```

9.5 JavaScript优化

代码分割

Code block

```

1  // webpack.config.js
2  module.exports = {
3      entry: {
4          main: './src/index.js',
5          vendor: './src/vendor.js'
6      },
7      optimization: {
8          splitChunks: {
9              chunks: 'all',
10             cacheGroups: {
11                 vendor: {
12                     test: /[\\/]node_modules[\\/]/,
13                     name: 'vendors',

```



```
14         chunks: 'all'
15     }
16 }
17 }
18 }
19 };
```

使用Web Workers处理复杂计算

Code block

```
1  // main.js
2  const worker = new Worker('worker.js');
3
4  worker.onmessage = function(event) {
5      console.log('计算结果:', event.data);
6  };
7
8  worker.postMessage({
9      numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
10     operation: 'sum'
11 });
12
13 // worker.js
14 self.onmessage = function(event) {
15     const { numbers, operation } = event.data;
16
17     let result;
18     if (operation === 'sum') {
19         result = numbers.reduce((sum, num) => sum + num, 0);
20     }
21
22     self.postMessage(result);
23 };
```

使用requestAnimationFrame实现流畅动画

Code block

```
1  function animate() {
2      // 更新动画
3      element.style.transform = `translateX(${position}px)`;
4      position += 2;
5
6      // 继续下一帧
7      requestAnimationFrame(animate);
8  }
9
10 // 开始动画
11 requestAnimationFrame(animate);
```

10. 师资&课程内容

1. 师资：一线大厂/外企现役P8级高级专家亲自带；
2. 内容：深度对标阿里P6-P7，涵盖大厂所有内训标准；

10.1 项目实战内容

github: <https://github.com/encode-studio-fe>

[前端实战课分享汇总](#)（可咨询班主任要密码权限）

10.2 服务内容

系统&突击两手抓

全程定制，学习计划+简历指导+面试指导+突击指导+全国内推+试用期全程陪跑，全网独家2V1模式，双师辅导（现役P8+HRD）



渠道：面试官直推/高管直推（简历直达leader）

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

10.5 近期报名学员案例

